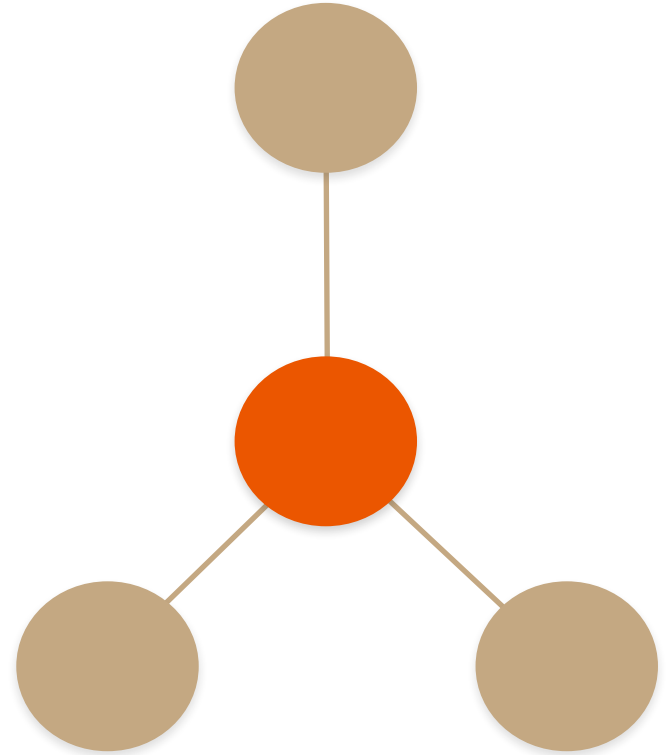


Dimensional Modelling - Build the Spine

*Theory - Facts, Dimensions, Grain, and
Denormalisation*





D2 - Dimensional Modelling

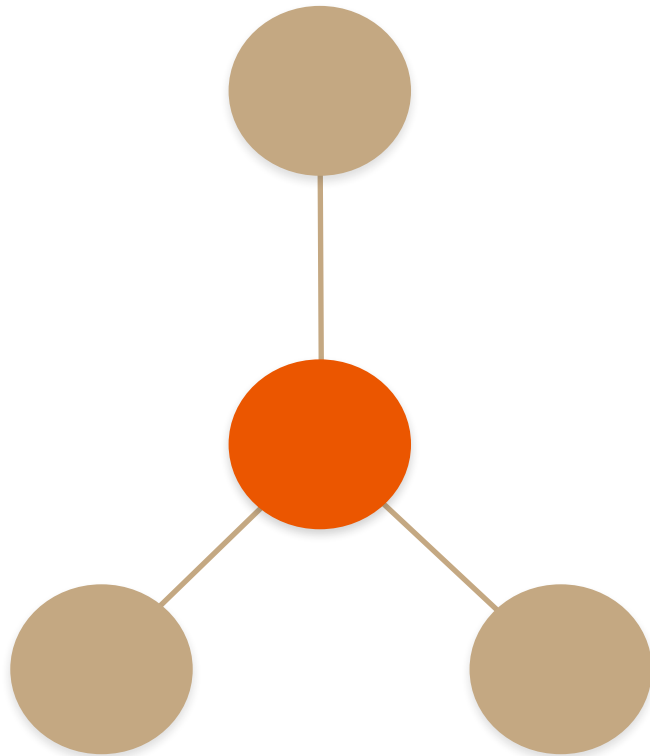


Theory - Facts, Dimensions, Grain, and Denormalisation | LO2

What You Will Be Able To Do

**From requirements to
a real schema.**

- Distinguish facts from dimensions in a real business process
- Declare a grain statement before naming a single column
- Read a star schema and explain why it is shaped that way
- Explain why a data warehouse denormalises on purpose



Recap - Where Day 1 Left Off



- **OLTP runs the business day to day; the warehouse supports decisions about the business**
- **You sketched three business questions the warehouse must answer**
- **You were shown a first glimpse of the star shape: one fact table, several dimensions**

Today: you design that star schema properly, then build it for real in the lab that follows.

Facts vs Dimensions



- A fact is a measurement you aggregate: COUNT, SUM, AVG
- A dimension is descriptive context you slice and group BY
- Test question: "enrollments BY course PER term" - what is being counted, and what is it grouped by?
- Answer: enrollment_count is the fact; course and term are dimensions

Facts and Dimensions in Our Spine



- `enrollment_count` and `grade` live inside `fact_enrollment`
- `course_name`, `department`, `semester`, and `nationality` are dimension attributes
- `enrollment_count` is an additive measure: it sums cleanly across any dimension you group by
- Additive measures are exactly what make star-schema queries fast and simple to write

KEY CONCEPT



Declare the Grain First

Before you name a single column, answer one question: what does one row in your fact table represent?

Get this wrong, and every query you write afterwards inherits the mistake.



Grain: The Level of Detail You Capture

- Grain is the exact business event that one fact row represents
- Our spine grain, stated exactly: "One row in fact_enrollment represents one student enrolled in one course during one academic term."
- Fine grain keeps every detail; coarse grain is compact but the detail is gone

Analogy: Camera Resolution

Familiar idea: choosing the resolution when you take a photo.

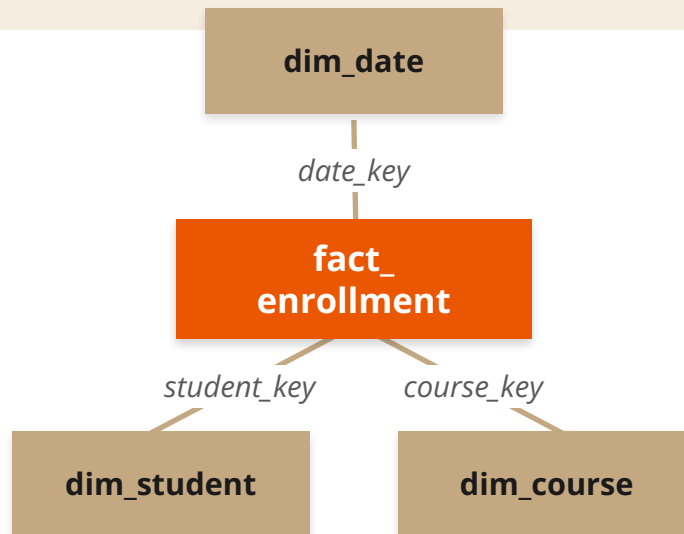
Mapping: fine-grain (one row per student per course per term) is like a high-resolution photo - you can zoom into one face in a crowd. Coarse grain (one row per course per term, students summed away) is like a low-resolution photo: smaller, quicker, but that detail was never captured.

Where it breaks: photo resolution is a smooth continuum of pixels; grain is a discrete design decision made up front, not a dial you nudge later.

Back to the warehouse: you can always downscale a high-res photo, but never upscale a low-res one. Likewise you can roll fine-grained facts up to a summary, but never drill a coarse fact back down to detail that was never stored.

The Star Schema

- One fact table sits at the centre of the schema
- Every dimension connects directly to the fact, and only to the fact
- Dimensions never join to one another in a pure star



Analogy: The Solar System

Familiar idea: the solar system.

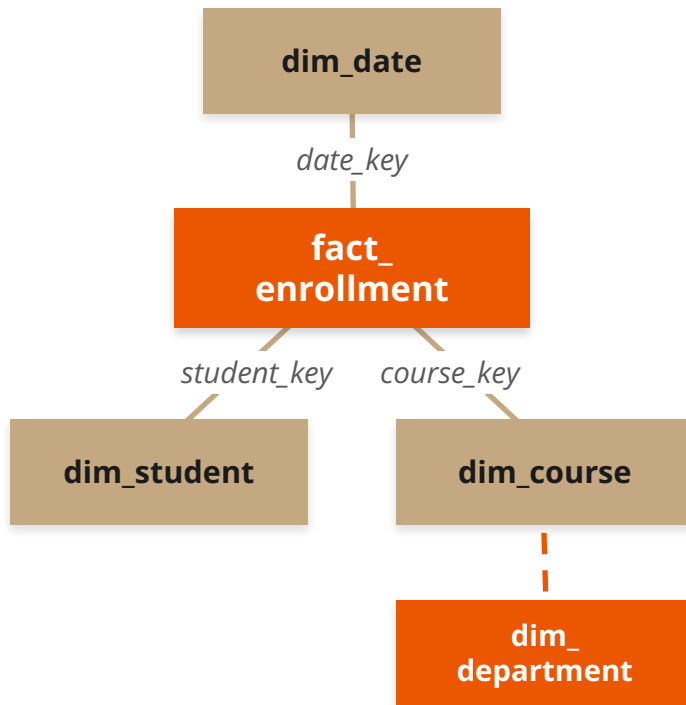
Mapping: the Sun at the centre is the fact table; every planet orbits it, just as each dimension connects to the fact. Crucially, planets do not orbit each other - and in a star schema, dimensions likewise connect only to the fact, never to one another.

Where it breaks: the Sun dwarfs the planets by mass, but a fact table is not "more important" - it usually just holds far more rows. An orbit is a continuous force; a foreign key is a discrete lookup.

Back to the warehouse: the schema is literally called a "star" because of this shape - one fact at the centre, dimensions radiating out, each exactly one join away from the numbers you are analysing.

The Snowflake Schema

- A snowflake normalises a dimension into further sub-dimensions
- Example: dim_course splits into dim_course + dim_department
- Department no longer repeats on every course row
- The trade: less redundancy, but an extra join to reach it



*dim_course keeps a department_key;
department attributes move one hop away*

Star vs Snowflake: The Trade-off

Star Schema

What you gain

- Fewer joins per query
- Simple, fast reads
- Easy for students and analysts to reason about

What you accept

- Repeated attribute values, e.g. department on every course row

Snowflake Schema

What you gain

- Less redundant storage
- One place to update a shared attribute

What you accept

- More joins per query
- More tables to design, build, and maintain

Try both live: the [HTML star schema explorer](#) has a Star / Snowflake toggle that shows the join count change in real time.

Why We Denormalise on Purpose

- OLTP normalises to avoid update anomalies - it is write-heavy
- The warehouse denormalises because it is read-heavy and write-rare
- Per Inmon's non-volatile principle from Day 1: once loaded, warehouse data is rarely changed in place
- Redundancy that would be risky in OLTP is safe, deliberate design in a DW - this is the heart of LO2

Analogy: Phone Contacts

Familiar idea: the contacts app on your phone.

Mapping: each contact card stores the company name in full, directly on the card. If fifty contacts work there, that name is stored fifty times - redundant, but your phone deliberately does this so opening a contact shows the company instantly, with no lookup.

Where it breaks: if the company renames itself, a normalised design updates one row; your phone must update all fifty cards. That is the real cost of denormalisation - reads become simple, updates become expensive.

Back to the warehouse: a star dimension stores attributes flatly and redundantly, exactly like a contact card - dim_course carries department on every row rather than pointing to a separate table. Because the warehouse is read-optimised and loaded in batches, the expensive-update cost barely ever bites, so the read simplicity is pure gain.

Worked Example, Step 1: What Are We Measuring?



- Start from your Day 1 questions - they all name one business process: **enrolment**
- Ask: what event are we measuring, and what triggers one record of it?
- The event: a student enrolls in a course, in a given term
- That event is the candidate grain for our fact table

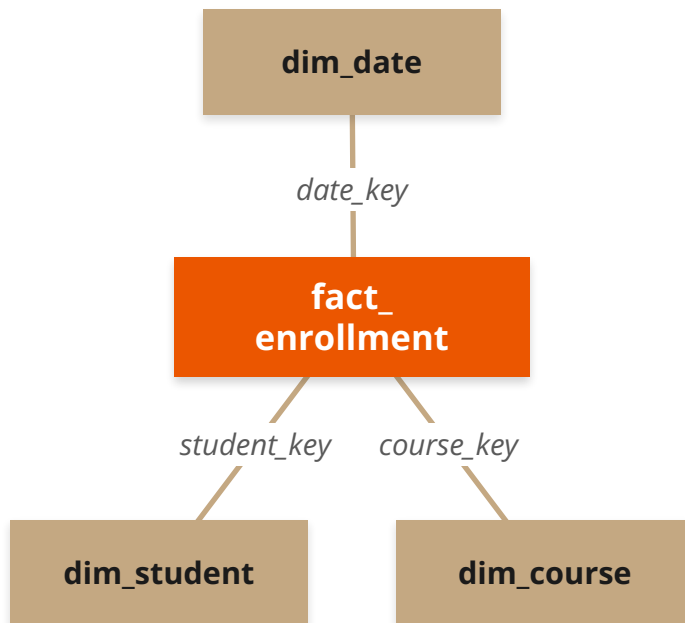
Worked Example, Step 2: Declare the Grain



- Grain: one student, one course, one term - stated before any column exists
- who -> dim_student | what -> dim_course | when -> dim_date
- Each "who / what / when" becomes a dimension the fact will reference
- Naming the dimensions falls out naturally once the grain is fixed first

Worked Example, Step 3: Assemble the Star

- **Facts:** enrollment_count (additive) and grade (outcome, filled in later)
- The three dimension keys sit on the fact row as foreign keys
- One fact table, three spokes, one line to each dimension
- This exact shape is what you create in SQL in the lab that follows



Common Modelling Mistakes



- **Mixing two different grains in one fact table**
- **Hiding a measure inside a dimension instead of the fact**
- **Over-snowflaking: normalising dimensions that never needed it**
- **Forgetting the date dimension entirely**
- **Choosing a grain too coarse to answer the business question**
- **Joining one dimension directly to another dimension**

Star or Snowflake? Your Call

**Would you ever choose snowflake over star?
When is the space saving worth the extra joins?**

Think about a dimension with millions of rows and heavy repetition.

5 minutes, then share.

Bridge to the Lab

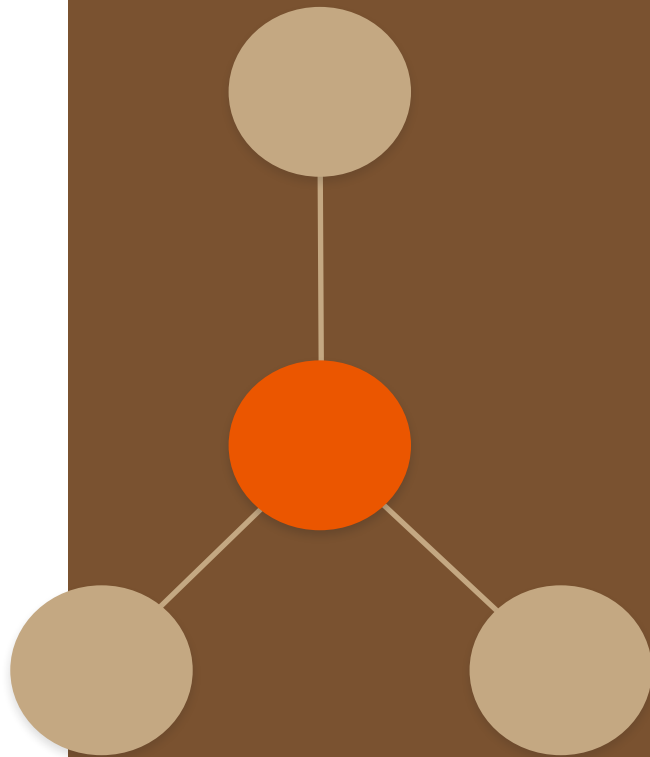


- You have designed the star: one fact, three dimensions, a stated grain
- Now you build it for real, in SQL, in PostgreSQL
- Deliverable: the running DDL, a DBeaver ERD showing the star shape, and one query result

Next: create the tables, load a seed dataset, and run your first analytical query.

Questions & Discussion

Day 2 Theory - Dimensional Modelling



Disclaimer & Copyrights



The information presented in this presentation is intended for general informational purposes only. It is not meant to be comprehensive or to constitute professional advice. Any reliance you place on such information is strictly at your own risk. While I strive to keep the information accurate and up-to-date, I do make no representations or warranties of any kind, express or implied, about the completeness, accuracy, reliability, suitability, or availability concerning the content within this presentation.

This presentation may include references or links to third-party websites or content. I do not endorse the views expressed or the information provided on such external sites. I am not responsible for the content of any linked site or any link contained in a linked site.

Any action you take upon the information presented in this presentation is strictly at your own discretion. I will not be liable for any losses or damages in connection with the use of this information.

This disclaimer is subject to change without notice.

@ Images and artworks used in this presentation are used for general informational purposes only. It may contain copyrighted material, the use of which may not have been specifically authorised by the copyright owner. This material is available in an effort to explain issues relevant to the topic. This should constitute a 'fair use' of any such copyrighted material.

Last modified: 5 Jul 2026